USER MANUAL

# R U OK?

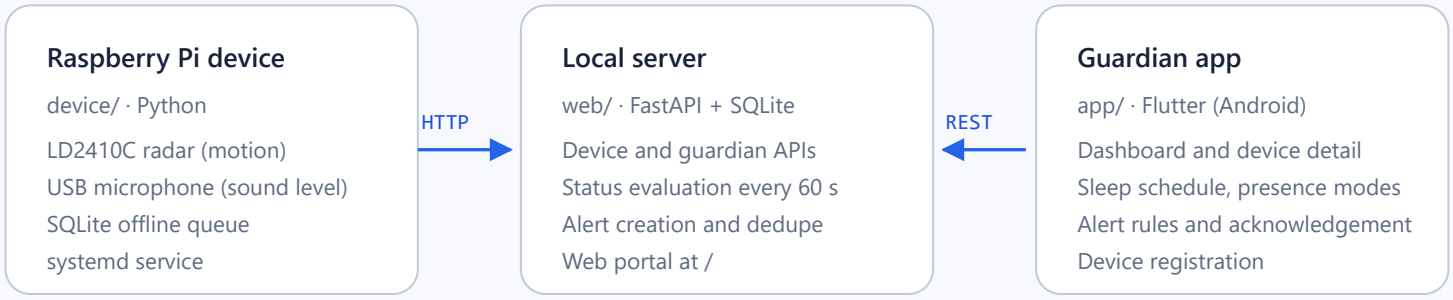
A wellness check-in system for people who live alone. A small sensor box in the home reports motion and sound to a local server; guardians watch status and receive alerts in a mobile app or web portal.

Version **0.2.0** Prepared **2 September 2026** Platforms **Windows server · Android app · Raspberry Pi device**

---

## 1 System overview

R U OK? answers one question for a guardian: *has there been any sign of life in the home recently?* It does this without cameras and without recording audio. The device only reports **that** motion or sound occurred and **when**.




`heartbeat · motion_detected · voice_detected → POST /api/device-events`

### Roles

| | |
|---|---|
| Monitored person | Lives with the device. Needs to do nothing. |
| Guardian | Family member or carer. Registers devices, sets schedules and thresholds, receives and acknowledges alerts. |
| Operator | Runs the server on a Windows PC on the home network and installs the device. |

## 2 Requirements

| COMPONENT | NEEDS |
| --- | --- |
| Server | Windows 10/11 with Python 3.11 or newer and PowerShell. Roughly 50 MB of disk. Port 8787 open on the local network. |
| Guardian app | Android 8.0 (API 26) or newer. To build from source: Flutter 3.35 with the Android SDK. |
| Device | Raspberry Pi Zero 2 WH (or any Pi with Wi-Fi) running Raspberry Pi OS, Python 3.9+, an HLK-LD2410C radar module and a USB microphone. Mock mode needs no sensors. |
| Network | Server, phone and Raspberry Pi on the same Wi-Fi network. The server uses plain HTTP; it is intended for a home LAN, not the public internet. |

## 3 Quick start

The order matters: the server must exist before the app can sign in, and a device must be registered in the app before the Raspberry Pi can report.

1. **Start the server**
   On the Windows PC run `web\setup_server.ps1` once, then `web\run_server.ps1` with `RUOK_HOST=0.0.0.0`. Note the PC's IP address from `ipconfig`.

2. **Install the guardian app**
   Install the APK on the phone. Open *Server settings*, enter `http://<PC IP>:8787`, and tap *Save and test connection*.

3. **Create a guardian account**
   Tap *New here? Create an account*, enter a name, email and password. Registration signs you in immediately.

4. **Register a device**
   Tap *Add device*. Copy the `device_uid` and `device_token`. The token is shown only once.

5. **Set up the Raspberry Pi**
   Copy the `device/` folder to the Pi, put the server URL, UID and token into `config/config.json`, and run `python main.py`. Start in *mock* sensor mode to verify the connection.

6. **Watch the dashboard**
   Within a minute the device shows **Normal** with a recent heartbeat. Set a sleep schedule and adjust thresholds if the defaults do not fit the household.

# 4 Server

The server is a FastAPI application backed by a single SQLite file. It stores accounts, devices, events and settings, evaluates every device once a minute, and serves both the device API and the guardian API. It also hosts a small web portal.

## 4.1 Install and run

```
cd D:\2026_project\ruok\web
.\setup_server.ps1        # creates .venv, installs FastAPI + uvicorn, initialises the database
.\run_server.ps1          # starts on http://127.0.0.1:8787
```

Configuration is read from environment variables. Copy `.env.example` or set them in the shell before starting:

| VARIABLE | DEFAULT | MEANING |
|---|---|---|
| `RUOK_HOST` | `127.0.0.1` | Bind address. Use `0.0.0.0` so phones and devices on the LAN can connect. |
| `RUOK_PORT` | `8787` | TCP port. |
| `RUOK_DB` | `data/ruok_server.sqlite` | Path of the SQLite database. Created on first start. |
| `RUOK_SECRET_KEY` | built-in dev value | Key used to sign guardian session tokens. **Set a long random value** for any use beyond a single developer's machine. |

Confirm the server is up by opening `http://127.0.0.1:8787/api/health` in a browser. It returns `{"ok": true, "service": "ruok-local-server", ...}`.

## 4.2 Reaching it from a phone

```
$env:RUOK_HOST = "0.0.0.0"
$env:RUOK_PORT = "8787"
.\run_server.ps1
ipconfig     # find the IPv4 address of the Wi-Fi adapter, e.g. 192.168.1.25
```

If Windows Defender Firewall prompts on first start, allow access on private networks. If the phone still cannot connect, add an inbound rule for TCP port 8787.

> **USB ALTERNATIVE FOR TESTING**
>
> With the phone connected by USB and USB debugging enabled, run `adb reverse tcp:8787 tcp:8787` and set the app's server address to `http://127.0.0.1:8787`. No Wi-Fi is needed.

### 4.3 Web portal

Open `http://<PC IP>:8787/` in a browser. The portal offers the same functions as the app: register or sign in, create devices, view the status grid, set sleep schedules and presence modes, edit alert rules, view events and acknowledge alerts. A button labelled *Evaluate alerts now* runs the status check immediately instead of waiting for the 60-second loop.

## 5 Guardian app

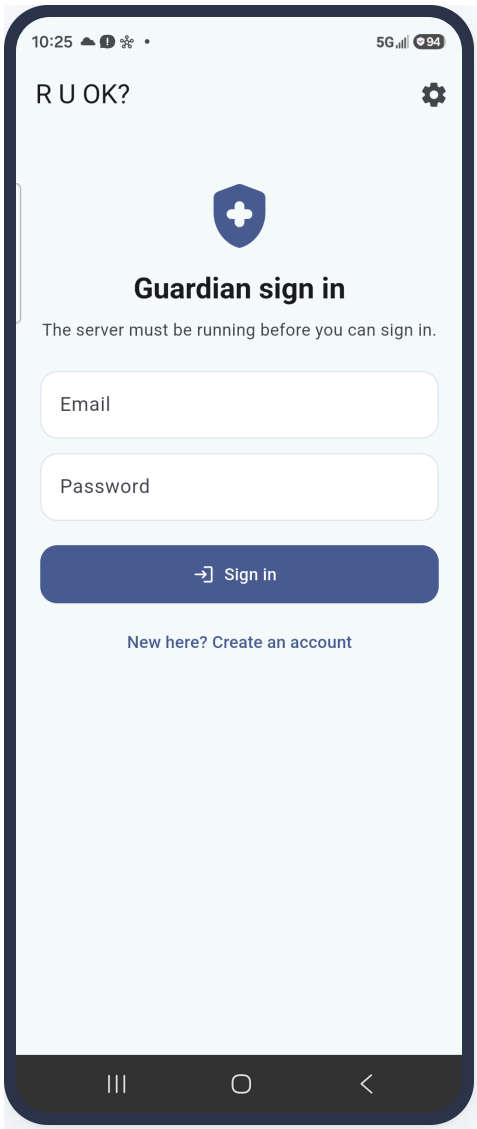
The Android app is the guardian's daily view. It keeps you signed in between launches and refreshes on pull-down.

### 5.1 Install and connect

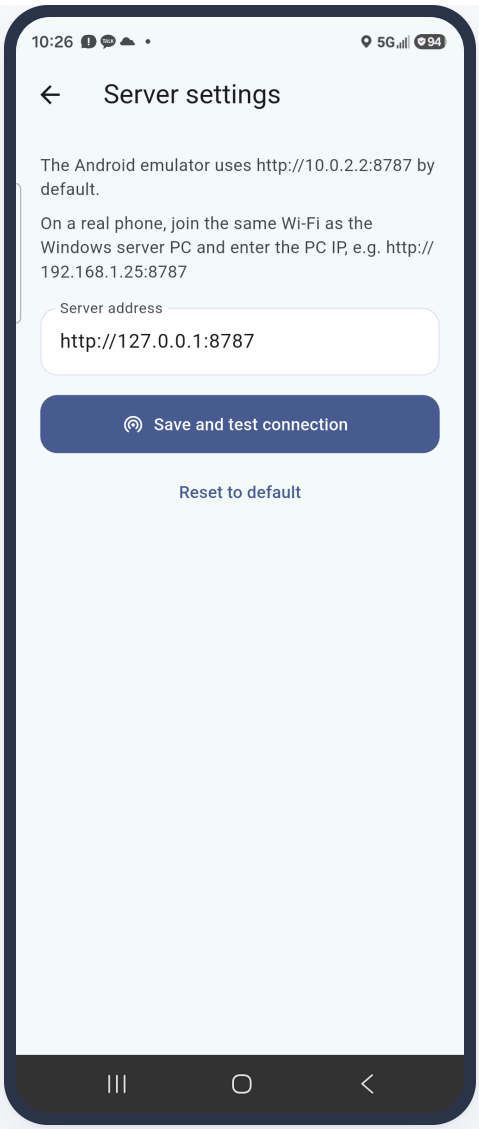
Install the debug APK ( `app-debug.apk` ) from the project, or build it yourself:

```
cd app
flutter pub get
flutter build apk --debug
# build\app\outputs\flutter-apk\app-debug.apk
```

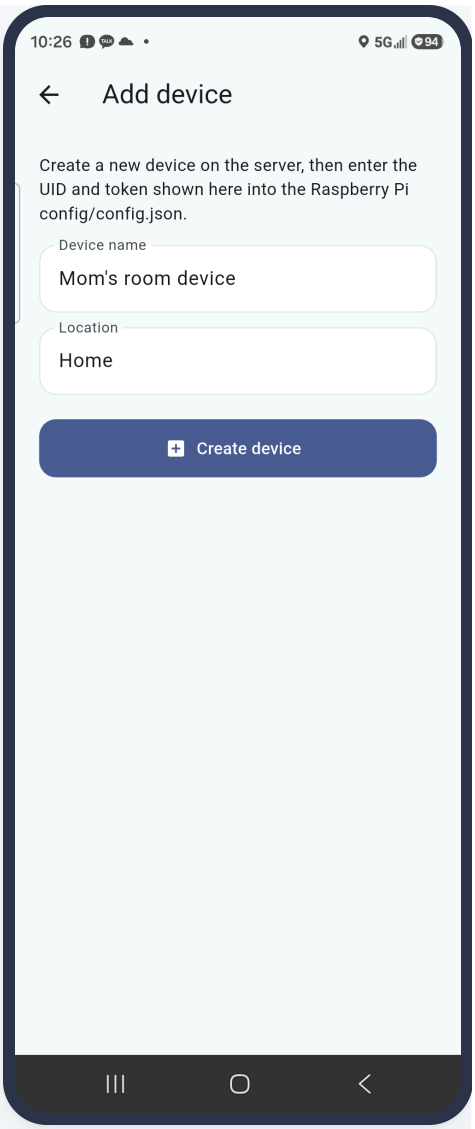
The first screen is *Guardian sign in*. Before signing in, tap the gear icon to open *Server settings* and point the app at your server. The default `http://10.0.2.2:8787` only works inside the Android emulator.




**Sign in**
Switch to account creation with the link below the button.




**Server settings**
Enter the PC address, then *Save and test connection*.

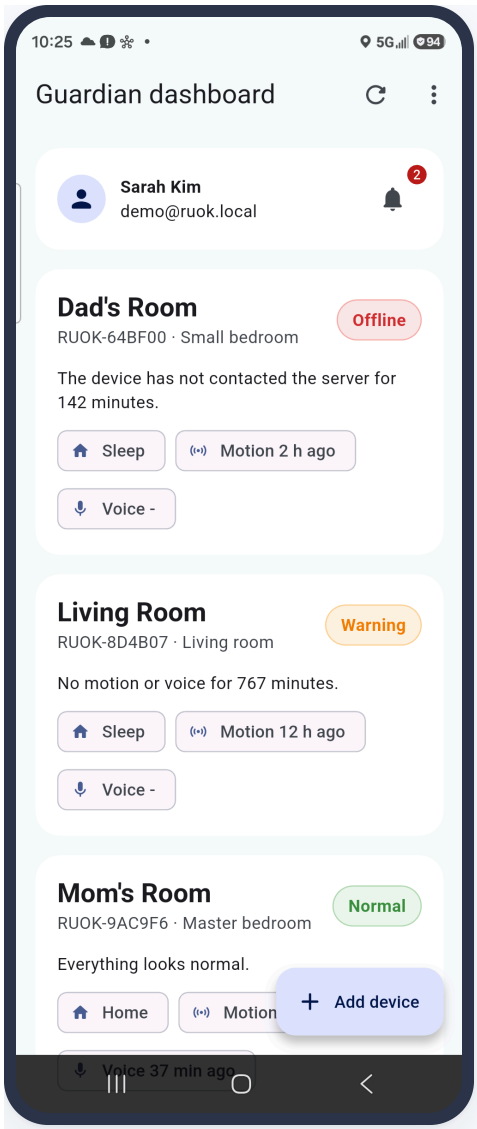



**Add device**
Creates the device on the server and shows its UID and one-time token.

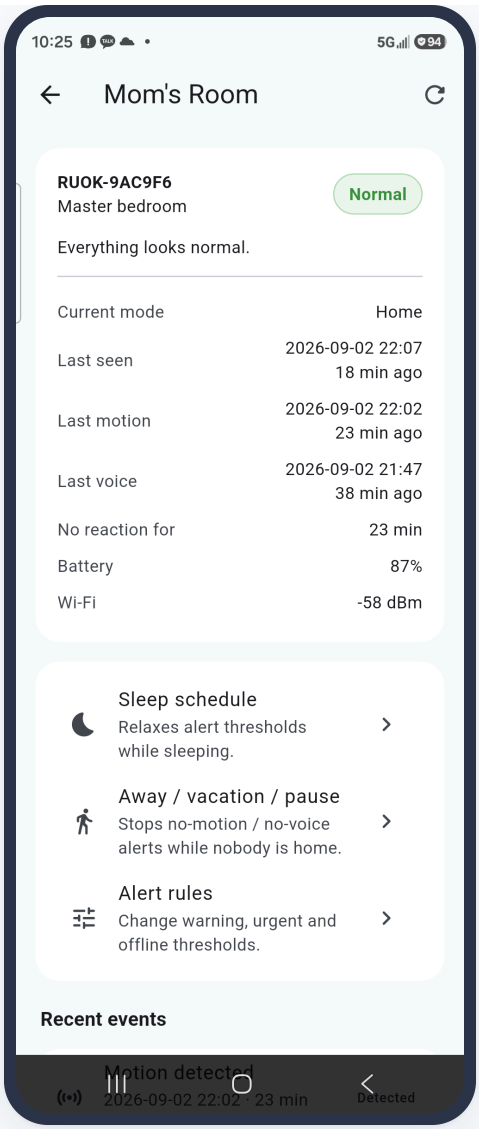
> **KEEP THE TOKEN**
>
> The `device_token` is displayed once, right after creation. If it is lost, delete the device and create a new one.

## 5.2 Dashboard

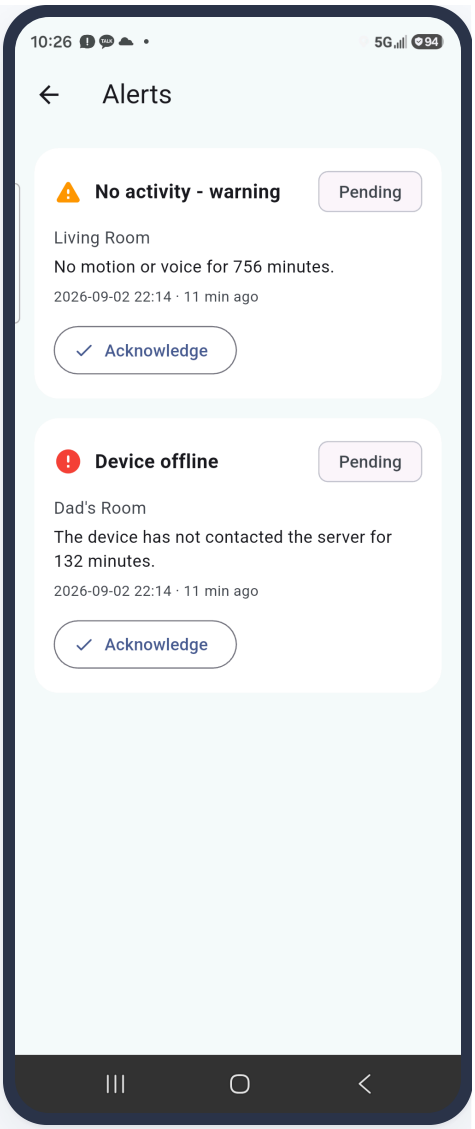
Every registered device appears as a card with a coloured status badge, the server's one-line reason, the active mode, and how long ago motion and sound were last detected. The bell in the header shows the number of unacknowledged alerts, and the five most recent alerts are listed below the devices.




**Dashboard**
Three devices in offline, warning and normal states.




**Device detail**
Timestamps, battery, Wi-Fi signal, settings and the event log.




**Alerts**
Pending alerts can be acknowledged with one tap.

The menu in the top-right corner opens *Server settings* and *Sign out*. Signing out clears the stored session on the phone only.

## 5.3 Device detail

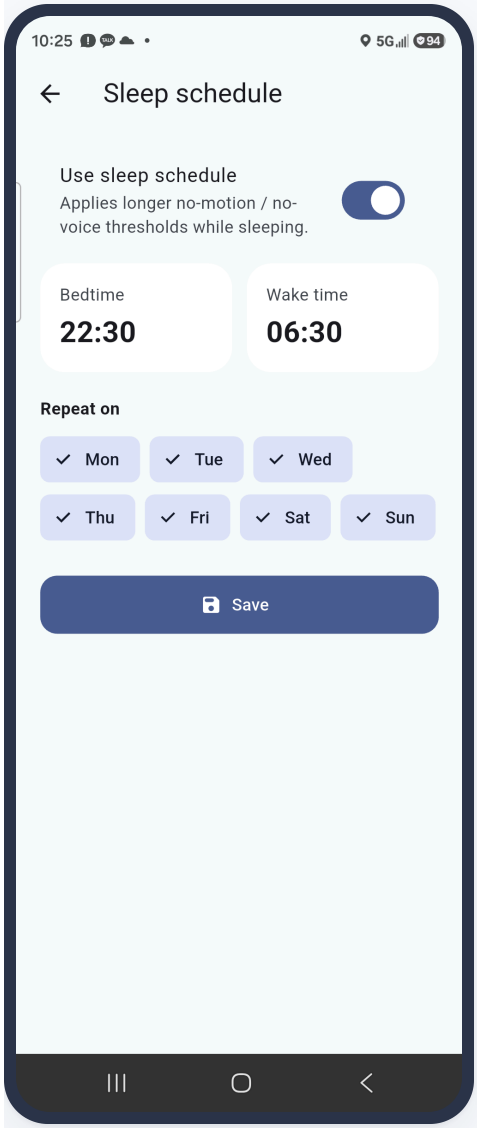
Tap a device card to open it. The top card shows the status the server computed for this device right now:

| ROW | MEANING |
|---|---|
| Current mode | *Home*, *Sleep* (inside the sleep schedule), or an active presence mode such as *Away*. |
| Last seen | Last time the device contacted the server for any reason, including heartbeats. |

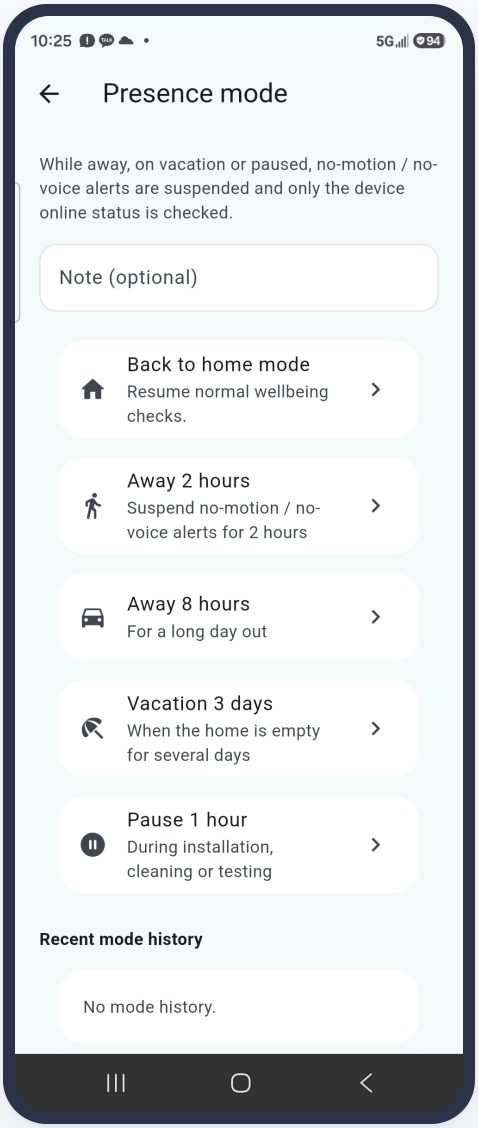
| | |
|---|---|
| Last motion / Last voice | Last detection of each kind, with a relative time. |
| No reaction for | Minutes since the most recent motion, sound or button press. This is the number compared against the thresholds. |
| Battery, Wi-Fi | Values the device reports in its heartbeat metadata, if configured. |

Below it are the three settings screens and the recent event log (motion, voice, heartbeat, button).

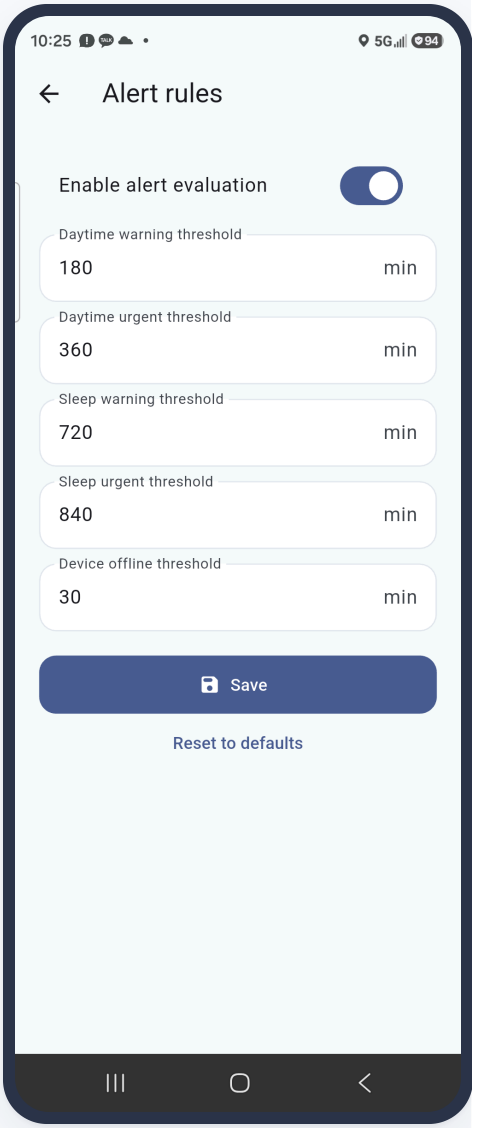
## 5.4 Sleep schedule, presence modes and alert rules




**Sleep schedule**
Tap a time to pick it; select the weekdays it applies to.




**Presence mode**
One-tap presets with an optional note; history below.




**Alert rules**
Thresholds in minutes; *Reset to defaults* restores the factory values.

### Sleep schedule

While the schedule is active, the server uses the longer *sleep* thresholds so that a full night without motion does not raise an alert. A schedule that crosses midnight (for example 22:30 to 06:30) is handled correctly. New devices start with 22:00 to 07:00 every day.

**Presence modes**

Use a preset when the person is out of the house. During *Away*, *Vacation*, *Pause* and *Maintenance* the server stops raising no-activity alerts and only checks that the device is online. Presets expire automatically after their duration; *Back to home mode* ends them early. If several modes overlap, the most restrictive wins: vacation, then away, then pause, then maintenance.

**Alert rules**

Five thresholds per device, all in minutes. Turning the switch off disables evaluation for that device entirely and shows **Alerts off** on the dashboard.

| RULE | DEFAULT | RAISES |
|---|---|---|
| Daytime warning threshold | 180 | **Warning** after this many minutes without motion, sound or a button press outside sleep time. |
| Daytime urgent threshold | 360 | **Urgent** outside sleep time. |
| Sleep warning threshold | 720 | **Warning** inside the sleep schedule. |
| Sleep urgent threshold | 840 | **Urgent** inside the sleep schedule. |
| Device offline threshold | 30 | **Offline** when the device has not contacted the server at all for this long. Checked before every other rule, including presence modes. |

## 5.5 Alerts

The server creates an alert when a device enters *Warning*, *Urgent* or *Offline*, and will not create a duplicate while one of the same type is still pending. Open the alert list from the bell icon and tap *Acknowledge* once you have checked on the person. Acknowledged alerts stay in the history with their status changed.

Alerts are visible in the app and the web portal only. This version does not send SMS, email or push notifications.

# 6 Raspberry Pi device

The device program is a single Python loop. Every second it polls the sensors; it sends a heartbeat on a timer, sends a motion or sound event when a detector fires (rate-limited by a minimum interval), and stores anything it could not deliver in a local SQLite queue that is flushed when the server is reachable again.

## 6.1 Hardware

Reference build: Raspberry Pi Zero 2 WH, HLK-LD2410C 24 GHz radar presence sensor on UART, and a USB microphone.

| LD2410C PIN | RASPBERRY PI PIN |
|---|---|
| VCC | 5 V |
| GND | GND |
| TX | GPIO 15 (RXD) |
| RX | GPIO 14 (TXD) |

Enable the hardware UART and disable the serial console:

```
sudo raspi-config
# Interface Options → Serial Port
#   Login shell over serial: No
#   Serial hardware enabled: Yes
```

Check the microphone is visible and records:

```
arecord -l
arecord -f S16_LE -c 1 -r 16000 -d 2 -t raw > /tmp/test.raw
```

## 6.2 Install and configure

```
cd ~/ruok_device_python           # the contents of the device/ folder
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt   # requests, pyserial
```

```
cp config/config.example.json config/config.json
nano config/config.json
```

Fill in the three values from the app's *Add device* screen and the server address:

```
{
  "server_url": "http://192.168.1.25:8787",
  "device_uid": "RUOK-9AC9F6",
  "device_token": "dev_...",
  "heartbeat_seconds": 60,
  "offline_flush_seconds": 20,
  "motion": { "enabled": true, "mode": "mock", "serial_port": "/dev/ttyS0",
              "baudrate": 256000, "min_interval_seconds": 30, "mock_probability": 0.25 },
  "voice":  { "enabled": true, "use_arecord": true, "threshold_rms": 500,
              "sample_seconds": 2, "interval_seconds": 30, "min_interval_seconds": 60 }
}
```

| KEY | MEANING |
|---|---|
| `heartbeat_seconds` | How often to tell the server the device is alive. Keep it well below the offline threshold (default 30 min). |
| `motion.mode` | `mock` generates random detections for testing without hardware; `ld2410` reads the radar over UART. |
| `motion.min_interval_seconds` | Minimum gap between two motion events sent to the server. |
| `voice.threshold_rms` | Loudness (RMS of 16-bit samples) above which a 2-second sample counts as sound. Raise it in a noisy room. |
| `battery_level`, `wifi_strength` | Optional numbers sent with each heartbeat and shown in the app. |

Run it in the foreground first and watch the log:

```
python main.py
# R U OK? device started. server=http://192.168.1.25:8787 uid=RUOK-9AC9F6
# Sent event: heartbeat
# Sent event: motion_detected
```

## 6.3 Run on boot

```
sudo bash system/install_service.sh
sudo systemctl enable ruok-device
```

```
sudo systemctl start ruok-device
sudo systemctl status ruok-device
journalctl -u ruok-device -f          # live log
```

The service restarts automatically five seconds after any crash and waits for the network to be online before starting.

## 7 How status is decided

Every 60 seconds, and whenever a device event arrives, the server evaluates each device in this order. The first matching rule sets the status.

| ORDER | CONDITION | STATUS |
|---|---|---|
| 1 | Alert rules switched off for this device | **Alerts off** |
| 2 | The device has never contacted the server | **Waiting** |
| 3 | No contact for ≥ offline threshold | **Offline** |
| 4 | A presence mode is active | **Away** **Vacation** **Paused** **Maintenance** |
| 5 | No motion, sound or button press ever recorded | **Waiting** |
| 6 | No reaction for ≥ urgent threshold (sleep or daytime set, depending on the schedule) | **Urgent** |
| 7 | No reaction for ≥ warning threshold | **Warning** |
| 8 | Inside the sleep schedule and within limits | **Sleeping** |
| 9 | Otherwise | **Normal** |

*Warning*, *Urgent* and *Offline* each create an alert, deduplicated against any pending alert of the same type for the same device. Because the offline check comes before presence modes, a device that stops reporting during a vacation still raises an alert. That is deliberate: the box should stay online even when the house is empty.

### Worked example

Default rules, sleep schedule 22:30 to 06:30, last motion at 21:00. At 00:00 the person has been still for 180 minutes. It is sleep time, so the 720-minute sleep warning applies and the status is **Sleeping**. If the

last motion had been at 09:00 the previous morning, 900 minutes have passed, which exceeds the 840-minute sleep urgent threshold: the status becomes **Urgent** and an alert is created.

## 8 API reference

All endpoints accept and return JSON. Guardian endpoints require `Authorization: Bearer <token>` from register or login. The device endpoint authenticates with two headers: `X-Device-UID` and `X-Device-Token`. Tokens expire after 7 days.

| METHOD | PATH | AUTH | PURPOSE |
|---|---|---|---|
| GET | `/api/health` | none | Liveness check. |
| POST | `/api/auth/register` | none | Create a guardian and their family. Body: name, email, password, phone (optional). |
| POST | `/api/auth/login` | none | Returns a session token and the user record. |
| GET | `/api/me` | guardian | Current user. |
| GET | `/api/devices` | guardian | Devices in the guardian's family, each with a `computed` status block. |
| POST | `/api/devices` | guardian | Create a device. Returns `device_uid` and the one-time `device_token`. |
| GET | `/api/devices/{id}/status` | guardian | Full evaluation for one device (see section 7). |
| GET | `/api/devices/{id}/events?limit=100` | guardian | Most recent events, newest first, up to 500. |
| GET / POST | `/api/devices/{id}/sleep-schedule` | guardian | Read or replace the schedule: start_time, end_time, days_of_week (0 = Monday), enabled. |
| GET | `/api/devices/{id}/presence-modes` | guardian | Last 50 presence-mode records. |
| POST | `/api/devices/{id}/presence-mode` | guardian | Set a mode: mode_type (home, away, vacation, pause, maintenance), start_at, end_at, note. *home* disables all active modes. |

| | | | |
|---|---|---|---|
| GET / POST | `/api/devices/{id}/alert-rules` | guardian | Read or update the five thresholds and the enabled flag. |
| GET | `/api/alerts` | guardian | Up to 200 alerts across the family, newest first. |
| POST | `/api/alerts/{id}/acknowledge` | guardian | Mark an alert acknowledged. |
| POST | `/api/admin/evaluate-alerts` | guardian | Run the evaluation loop immediately. |
| POST | `/api/device-events` | device | Report an event: event_type (heartbeat, motion_detected, voice_detected, button_pressed), detected, value_number, value_text, timestamp, meta. |

### Example: device heartbeat

```
POST /api/device-events
X-Device-UID: RUOK-9AC9F6
X-Device-Token: dev_...
Content-Type: application/json

{ "event_type": "heartbeat", "detected": true,
  "timestamp": "2026-09-02T22:07:14",
  "meta": { "battery_level": 87, "wifi_strength": -58 } }
```

# 9 Troubleshooting

| SYMPTOM | LIKELY CAUSE AND FIX |
|---|---|
| App shows *Cannot reach the server* | Server bound to 127.0.0.1 only, phone on a different network, or firewall. Start with `RUOK_HOST=0.0.0.0`, confirm the PC IP, allow port 8787. |
| App shows *No response from the server* | The address is reachable but nothing answers within 12 seconds. Check the server console and that the port matches. |
| Sign-in returns *Invalid or expired token* on the dashboard | Tokens last 7 days, and change if `RUOK_SECRET_KEY` changes. The app signs you out automatically; sign in again. |
| Device stays **Waiting** | The Pi has not sent anything. Check `config.json` values, run `python main.py` in the foreground and read the error. |

| | |
|---|---|
| Device log says *401 Unauthorized* | Wrong `device_uid` or `device_token`. Create a new device in the app and copy both values again. |
| Device goes **Offline** although it is running | Heartbeat interval longer than the offline threshold, or the Pi lost Wi-Fi. Events queue locally and are re-sent when the connection returns. |
| Sound is never detected | `arecord` not installed or wrong default capture device. Test with `arecord -l`; lower `threshold_rms`. |
| Motion is never detected in `ld2410` mode | Serial console still enabled, TX/RX swapped, or wrong port. Verify with raspi-config and try `/dev/serial0`. |
| Times in the app look several hours off | A database created by an older version stores creation times in UTC. Delete `web/data/ruok_server.sqlite` and run `python setup_db.py` (this removes all accounts and devices). |

# 10 Project layout

```
ruok/
  app/                    Flutter guardian app (Android)
    lib/core/             strings.dart (all UI text), labels, formatters, config
    lib/models/           Device, DeviceStatus, AlertItem, SleepSchedule, ...
    lib/services/         api_client.dart, local_store.dart
    lib/screens/          splash, auth, dashboard, device_detail, settings, ...
    lib/widgets/          AsyncBody, AlertCard, StatusBadge, InfoRow, BusyIcon
    android/              native project, applicationId com.ruok.guardian
    docs/screenshots/     the images used in this manual
    test/core_test.dart   unit tests (flutter test)
  web/                    FastAPI server
    app/main.py           routes
    app/logic.py          status evaluation and alert creation
    app/database.py       SQLite schema
    app/security.py       password hashing, session and device tokens
    web/index.html        guardian web portal
    setup_server.ps1, run_server.ps1, setup_db.py
  device/                 Raspberry Pi program
    main.py               event loop
    sensors/              motion_ld2410.py (mock + UART), voice_detector.py
    network/              api_client.py, offline_queue.py
    system/               install_service.sh, ruok-device.service
    config/config.example.json
```

Developer checks for the app: `flutter analyze` and `flutter test` both pass on the current source. To change the default server address at build time use `flutter run --dart-define=RUOK_BASE_URL=http://192.168.1.25:8787`.

# 11 Privacy and limitations

## Privacy by design

- No camera. The radar sensor reports presence and distance only.
- No audio is stored or transmitted. The microphone sample is reduced to a single loudness number on the device and discarded.
- The server records only event types and timestamps. All data stays on the guardian's own PC.

## Known limitations of this version

- Alerts appear in the app and portal only; there is no SMS, email or push delivery.
- The LD2410C driver in `ld2410` mode detects serial activity rather than decoding the full radar protocol. Mock mode is the fully verified path.
- Family membership is stored but every member currently has full access to every device in the family.
- The server speaks plain HTTP and is meant for a home network. Do not expose port 8787 to the internet without adding HTTPS and a strong `RUOK_SECRET_KEY`.
- A device token cannot be re-issued; create a new device if it is lost.